

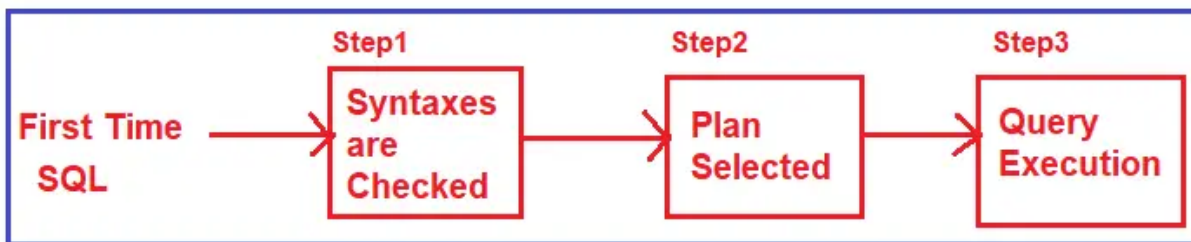
Stored Procedure in SQL Server with Examples

In this article, I am going to discuss **Stored Procedures in SQL Server** with Examples. As a SQL Server developer, it is the most important concept for you to understand. As part of this article, we are going to discuss the following pointers related to the Stored Procedure.

1. **Why do we need a Procedure in SQL Server?**
2. **What is a Stored Procedure in SQL Server?**
3. **How can we create a Stored Procedure?**
4. **How to call a Stored Procedure in SQL Server?**
5. **Multiple Examples to understand SQL Server Stored Procedure.**
6. **How to view the text of a Procedure in SQL Server?**
7. **How to Drop and Alter a Procedure in SQL Server?**
8. **Different Types of Parameters in SQL Server Stored Procedure**
9. **Understanding the Input Parameters in SQL Server Procedure**
10. **Understanding the SQL Server Stored Procedure Output Parameters**
11. **What are the advantages of using a Stored Procedure?**
12. **What is an execution plan?**

Why do we need a Stored Procedure in SQL Server?

Before going to understand why we need a Stored Procedure, let us first understand what happens when we execute a simple SQL statement in SQL Server. When any SQL Statements are fired on SQL Server, then three steps are happening in order which is shown in the below image.

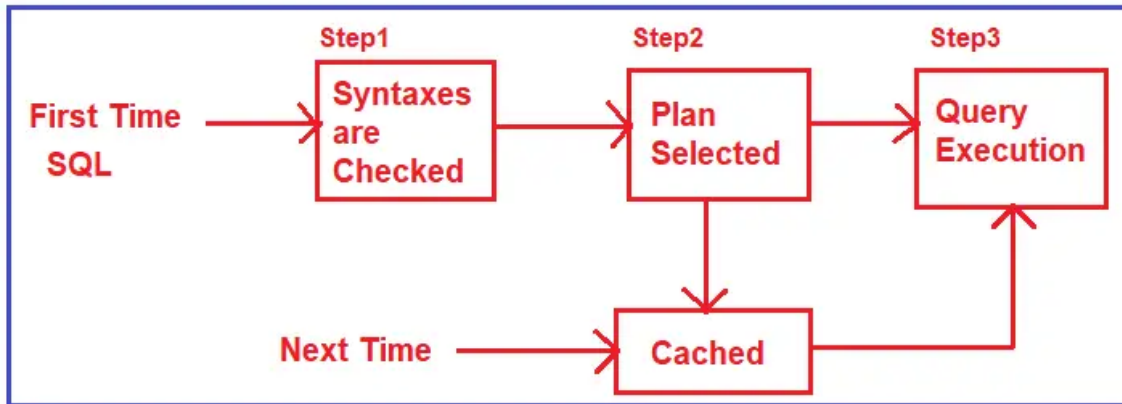


1. **Syntax Checked:** This step ensures that the syntaxes are correct and there is no error and the query is ready for executing on SQL Server.
2. **Plan Selected:** Once the syntaxes are checked, the second step is to select a plan. The SQL Query must be using some tables. It will go and check what types of indexes are exists on these tables, it will also check can use these indexes or a table scan is fine. So, the second step is to select a proper execution plan to execute the query.
3. **Query Execution:** Once the plan is selected, the final step is to execute the query and the output is seen by the end-user.

So, any SQL Statement fire on SQL Server should go through these three steps.

Now somehow, if we ensure that the first two steps (i.e. Syntax Checked and Plan Selected) are executed only once, would not it be great. In other words, the first time the SQL is executed, the syntaxes are checked, the execution plan is selected and the execution plan is cached in memory. So, if the same SQL statements are fired again, then these two steps are not going to be executed, rather the execution plan is taken from the

cache and executed and that will definitely increase the performance of the application which is shown in the below image.



This is what exactly the stored procedure does in SQL Server. When we create a stored procedure, the syntaxes are checked while creating the procedure or we can say at the design. When we execute the procedure for the first time, the best execution plan is selected and is cached in memory. And after that whenever we call the stored procedure, the query execution plan is taken from the cache rather than creating again and again and executed.

There are also other advantages of using stored procedures which we will discuss in our upcoming articles. With this keep in mind, let us proceed and understand the SQL Server Stored Procedure in detail.

What is a Stored Procedure in SQL Server?

A SQL Server Stored Procedure is a database object which contains pre-compiled queries (a group of T-SQL Statements). In other words, we can say that the Stored Procedures are a block of code designed to perform a task whenever we called.

How can we create a Stored Procedure in SQL Server?

In SQL Server, you can create a stored procedure by using the **CREATE PROCEDURE** or **CREATE PROC** statement. Again, you can create a procedure with or without parameters. Please have a look at the below image for the **Syntax of Stored Procedure**.

```

-- Syntax for creating a procedure without parameter
CREATE/ALTER PROCEDURE ProcedureName
AS
BEGIN
    Procedure Body
END

-- Syntax for creating a procedure with parameters
CREATE/ALTER PROCEDURE ProcedureName
(
    @Param1 DataType,
    @Param2 DataType,
    @Paramn DataType,
)
AS
BEGIN
    Procedure Body
END

```

A stored procedure is very much similar to a function in C, C++ languages or a method in Java or C# languages. The procedure definitions contain two parts in it

1. **Procedure header**
2. **Procedure body**

The content above “**AS**” is known as the procedure header and the content below the “**AS**” is known as the procedure body. If required we can pass the parameter to a procedure to make the procedure more dynamic.

How to call a Stored Procedure in SQL Server?

Once we create the stored procedure, then it is physically stored on the server as a “**database object**” which can be called from anywhere connecting to the server. We can call the procedure from anywhere that is from a new query window or from any application that is developed using java or .net language also in three different ways as shown in the below image.

```

EXECUTE ProcedureName VALUES
OR
EXEC ProcedureName VALUES
OR
ProcedureName VALUES

```

Note: Another way to execute a stored procedure is to right-click on the procedure name in object explorer and select “**Execute Stored Procedure**”.

Stored Procedure in SQL Server Without Parameter

The following stored procedure simply print a welcome message on the screen

```
CREATE PROCEDURE spDisplayWelcome
AS
BEGIN
    PRINT 'WELCOME TO PROCEDURE in SQL Server'
END
```

Calling a Stored Procedure:

```
EXECUTE spDisplayWelcome
Or
EXEC spDisplayWelcome
Or
spDisplayWelcome
```

Output: **WELCOME TO PROCEDURE in SQL Server**

Note: While naming the user-defined stored procedures we should not have to use “sp_” as a prefix as it is recommended by Microsoft. The reason is all the system-defined procedures in SQL Server are prefixed with “sp_”. So to avoid the ambiguity between the user-defined stored procedure and the system stored procedures and for any conflicts with some future coming system procedure we should not use sp_ as a prefix to our user-defined stored procedure.

Let’s see another example where we will fetch the data from a database table.

We are going to use the following Employee table.

ID	Name	Gender	DOB	DeptID
1	Pranaya	Male	1996-02-29 10:53:27.060	1
2	Priyanka	Female	1995-05-25 10:53:27.060	2
3	Anurag	Male	1995-04-19 10:53:27.060	2
4	Preety	Female	1996-03-17 10:53:27.060	3
5	Sambit	Male	1997-01-15 10:53:27.060	1
6	Hina	Female	1995-07-12 10:53:27.060	2

Please use the following SQL Script to create and populate the Employee table with the required sample data.

```
-- Create Employee Table
CREATE TABLE Employee
(
    ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Gender VARCHAR(50),
    DOB DATETIME,
    DeptID INT
)
GO
```

```
-- Populate the Employee Table with test data
INSERT INTO Employee VALUES(1, 'Pranaya', 'Male', '1996-02-29 10:53:27.060', 1)
INSERT INTO Employee VALUES(2, 'Priyanka', 'Female', '1995-05-25 10:53:27.060', 1)
INSERT INTO Employee VALUES(3, 'Anurag', 'Male', '1995-04-19 10:53:27.060', 2)
INSERT INTO Employee VALUES(4, 'Preety', 'Female', '1996-03-17 10:53:27.060', 1)
INSERT INTO Employee VALUES(5, 'Sambit', 'Male', '1997-01-15 10:53:27.060', 1)
INSERT INTO Employee VALUES(6, 'Hina', 'Female', '1995-07-12 10:53:27.060', 2)
GO
```

Create a stored procedure to get the names, gender, and the dob of all employees from the table Employee table.

```
CREATE PROCEDURE spGetEmployee
AS
BEGIN
    Select Name, Gender, DOB from Employee
END
-- To Execute the Procedure
EXEC spGetEmployee
```

When we execute the above statement it will give us the below output.

Name	Gender	DOB
Pranaya	Male	1996-02-29 10:53:27.060
Priyanka	Female	1995-05-25 10:53:27.060
Anurag	Male	1995-04-19 10:53:27.060
Preety	Female	1996-03-17 10:53:27.060
Sambit	Male	1997-01-15 10:53:27.060
Hina	Female	1995-07-12 10:53:27.060

How to View the text of a Stored Procedure in SQL Server?

Once you created the stored procedure and later if you want to view the text of the stored procedure then you need to use the **sp_helptext** system-defined stored procedure by supplying the procedure name as a parameter as shown below

Example: `sp_helptext spGetEmployee`

Else just right click on the stored procedure in object explorer Script procedure as Create To new query editor window

How to change the name and body of a stored procedure in SQL Server?

```
CREATE PROCEDURE spGetEmployee
```

```

AS
BEGIN
    SELECT Name, Gender, DOB FROM Employee
END
-- How to change the body of a stored procedure
-- User Alter procedure to change the body
ALTER PROCEDURE spGetEmployee
AS
BEGIN
    SELECT Name, Gender, DOB
    FROM Employee
    ORDER BY Name
END
-- To change the procedure name from spGetEmployee to spGetEmployee1
-- Use sp_rename system defined stored procedure
EXEC sp_rename 'spGetEmployee', 'spGetEmployee1'

```

How to Drop a Stored Procedure?

In order to drop a stored procedure, all you need to use the following syntax

DROP PROCEDURE ProcedureName

Example: Drop proc spGetEmployee1 or Drop Procedure spGetEmployee1

Different Types of Parameters in SQL Server Stored Procedure.

The parameters of a Stored Procedure in SQL Server can be of two types

1. **Input parameters**
2. **Output parameters**

The Input Parameters in SQL Server Stored Procedure are used for bringing the values into the procedure for execution. On the other hand, the Output Parameters are used to carrying a value out of the procedure after execution.

When a parameter is declared with the output keyword, then we only require assigning a value to the parameter inside the procedure so that the procedure will send that value out at the end of procedure execution.

Understanding the Input Parameters in SQL Server Procedure:

Let us understand the Input Parameters in SQL Server with an example. Let's create a procedure that will take two input integer parameters and then perform the sum operation and finally print the result.

Example: Stored Procedure for adding two variables value

```

-- Create a Procedure
ALTER PROCEDURE spAddTwoNumbers(@no1 INT, @no2 INT)
AS

```

```

BEGIN
    DECLARE @Result INT
    SET @Result = @no1 + @no2
    PRINT 'RESULT IS: ' + CAST(@Result AS VARCHAR)
END
GO
-- Calling the procedure:
EXECUTE spAddTwoNumbers 10, 20
-- OR
EXECUTE spAddTwoNumbers @no1=10, @no2=20
-- OR calling the procedure by declaring two variables as shown below
DECLARE @no1 INT, @no2 INT
SET @no1 = 10
SET @no2 = 20
EXECUTE spAddTwoNumbers @no1, @no2

```

Note: The Parameters and variables that we created must have an @ prefix in their name.

Example: Create a Procedure to get the employee information bypassing the employee gender and department id from the Employee table

```

CREATE PROCEDURE spGetEmployeesByGenderAndDepartment
    @Gender VARCHAR(20),
    @DeptID INT
AS
BEGIN
    SELECT Name, Gender, DOB, DeptID
    FROM Employee
    WHERE Gender = @Gender AND DeptID = @DeptID
END
GO

```

In order to invoke the above SQL Server Stored Procedure, we need to pass the value for @Gender and @DeptID input parameters. If we don't specify the name of the parameters we have to first pass the value for the @Gender parameter and then for the @DeptID parameter as shown below.

EXECUTE spGetEmployeesByGenderAndDepartment 'Male', 1

On the other hand, if we change the order, then we will get an error stating “**Error converting data type varchar to int.**” This is because the value of “**Male**” is passed into the @DeptID parameter. Since @DeptID is an integer, we get the type conversion error.

EXEC spGetEmployeesByGenderAndDepartment 1, 'Male'

When we specify the names of the parameters when executing the stored procedure the order doesn't matter. The example is given below.

EXEC spGetEmployeesByGenderAndDepartment @DeptID=1, @Gender='Male'

Note: While we are calling the stored procedure passing the values in order is very important. In the order they are declared in the procedure we need to pass the values in the same order. You can also pass the value in any order, but at that time you have to specify the variable name before the value to which you are passing the values.

Example: Create a procedure to update the Employee details in the Employee table based on the Employee id.

```
-- Create a Procedure
CREATE PROCEDURE spUpdateEmployeeByID
(
    @ID INT,
    @Name VARCHAR(50),
    @Gender VARCHAR(50), @DOB DATETIME,
    @DeptID INT
)
AS
BEGIN
    UPDATE Employee SET
        Name = @Name,
        Gender = @Gender,
        DOB = @DOB,
        DeptID = @DeptID
    WHERE ID = @ID
END
GO
-- Executing the Procedure
-- If you are not specifying the Parameter Names then the order is important
EXECUTE spUpdateEmployeeByID 3, 'Palak', 'Female', '1994-06-17 10:53:27.060',
-- If you are specifying the Parameter Names then order is not mandatory
EXECUTE spUpdateEmployeeByID @ID =3, @Gender = 'Female', @DOB = '1994-06-17
10:53:27.060', @DeptID = 3, @Name = 'Palak'
```

SQL Server Stored Procedure Output Parameters:

The Input parameters of SQL Server Stored Procedure are used for bringing the values into the procedure for execution. On the other hand, the SQL Server Stored Procedure output parameters are used for carrying a value out of the procedure after its execution. We only require assigning a value to the output parameter inside the procedure so that procedure will send that value out at the end of the procedure execution. The Output parameter in SQL Server can be declared either by using the **OUT** or **OUTPUT** keyword

Example: Stored Procedure with Output Parameter

Let's create a simple stored procedure to understand the SQL Server Stored Procedure Output Parameters

```
CREATE PROCEDURE spGetResult
    @No1 INT,
    @No2 INT,
```

```

    @Result INT OUTPUT
AS
BEGIN
    SET @Result = @No1 + @No2
END

```

The above SQL Server Stored Procedure takes 3 parameters. The **@No1** and **@No2** are input parameters by default whereas the **@Result** is the output parameter. The Parameter which is created using the **OUT** or **OUTPUT** keyword is called the output parameter in SQL Server.

To execute a procedure with output parameter, First, we need to declare a variable, then we need to pass that variable while calling the procedure by specifying the type as output as shown below.

```

-- To Execute Procedure
DECLARE @Result INT
EXECUTE spGetResult 10, 20, @Result OUT
PRINT @Result

```

When we execute the above code it will print 30.

Let's see more examples for a better understanding of SQL Server stored procedure output parameters.

Create a stored procedure to get the total number of employees in the Employee table by Gender. As we already discussed to create a Stored Procedure in SQL Server with an output parameter, we need to use the keyword **OUT** or **OUTPUT**. In the following Stored Procedure, the **@EmployeeCount** is an output parameter as we specified the parameter with the OUTPUT keyword.

```

CREATE PROCEDURE spGetEmployeeCountByGender
    @Gender VARCHAR(30),
    @EmployeeCount INT OUTPUT
AS
BEGIN
    SELECT @EmployeeCount = COUNT(ID)
    FROM Employee
    WHERE Gender = @Gender
END

```

Let's see the different ways to execute the above SQL Server Stored Procedure with the output parameter.

1. **Step1:** First declare a variable of the same data type as that of the output parameter. Here we have declared the **@EmployeeTotal** integer variable.
2. **Step2:** Then we need to pass the **@EmployeeTotal** variable to the stored procedure. We have to specify the variable with the **OUTPUT** keyword. If we don't specify the **OUTPUT** keyword, the variable will be **NULL**.
3. **Step3.** Execute

Way1: Allowed

```

DECLARE @EmployeeTotal INT
EXECUTE spGetEmployeeCountByGender 'Male', @EmployeeTotal OUTPUT
PRINT @EmployeeTotal

```

Note: If we don't specify the output keyword when executing the stored procedure then the **@EmployeeTotal** value will be null. For example, see the following

```

DECLARE @EmployeeTotal INT
EXECUTE spGetEmployeeCountByGender 'Male', @EmployeeTotal
PRINT @EmployeeTotal

```

Whether it will print null or not check the following:

```

DECLARE @EmployeeTotal INT
EXECUTE spGetEmployeeCountByGender 'Male', @EmployeeTotal
IF (@EmployeeTotal IS NULL)
    PRINT '@EmployeeTotal IS NULL'
ELSE
    PRINT '@EmployeeTotal IS NOT NULL'

```

Way2: Not Allowed

```

DECLARE @EmployeeTotal INT
EXECUTE spGetEmployeeCountByGender @EmployeeTotal OUTPUT, 'Male'
PRINT @EmployeeTotal

```

Way3: Allowed

We can pass the parameters in any order when we use the parameter names. Here, we are first passing the output parameter and then the input **@Gender** parameter.

```

DECLARE @EmployeeTotal INT
EXECUTE spGetEmployeeCountByGender @EmployeeCount = @EmployeeTotal OUTPUT, @Gender = 'Male'
PRINT @EmployeeTotal

```

Stored Procedure with Default Values:

Let's see an example of how to use the stored procedure with default values.

```

CREATE PROCEDURE spAddNumber(@No1 INT= 100, @No2 INT)
AS
BEGIN
    DECLARE @Result INT
    SET @Result = @No1 + @No2
    PRINT 'The SUM of the 2 Numbers is: ' + CAST(@Result AS VARCHAR)
END
-- Executing the above procedure:
1. EXEC spAddNumber 3200, 25

```

2. **EXEC** spAddNumber @No1=200, @No2=25
3. **EXEC** spAddNumber @No1=DEFAULT, @No2=25
4. **EXEC** spAddNumber @No2=25

In the 3rd and 4th cases, it uses the default value of 100 to the variable @No1 which has been given while creating the procedure.

What are the advantages of using a Stored Procedure in an SQL Server?

This is one of the most frequently asked interview questions in SQL Server. Let discuss this question in detail.

Execution Plan Retention which Improves the Application Performance

As there is no unnecessary compilation of queries this will reduce the burden on the database (when we send a query to a SQL Server three things happen in order, 1st it checks the syntax of that query, 2nd it compiles that query, 3rd it generates an execution plan) as response user will get a quick response. Let's get into more details.

The Stored Procedures are pre-compiled and their execution plan is cached and used again when the same stored procedure is executed again. Although ad-hoc queries also create and reuse plans, the plan is reused only when the query is the textual match and the datatypes are matching with the previous call. Any changes in the datatype or you have an extra space in the query then, a new plan is created.

Reduces the Network Traffic

The Stored Procedure reduces network traffic. When we execute a stored procedure we need to send the procedure name and parameters so only these things are passed on the network but if we are not using the stored procedure then we need to write the ad-hoc queries and we need to execute them which may contain many numbers of lines. So the stored procedure reduces the network traffic as a result performance of the application increase.

Code Re-usability and Better Maintainability

Multiple applications can use the same stored procedure. The different applications which want similar kind of data then they can use the same stored procedure. The advantage is that if we want to change the stored procedure then we need to change it in one place that will affect to all the application that uses it whereas if it is inline SQL query and if we have to use it in multiple applications, then we end up with multiple copies of the same inline SQL query, and if the logic has to change, then we have to change the logic at all the places, which makes it harder maintaining inline SQL. So, the stored procedure provides code reusability and maintainability.

Better Security

By granting permission to the underlying database the user can do everything. He can view all the records as well as modify the records. But if we want to restrict the user only to view the records then we need to grant only for that stored procedure which will display the records. In that way, we achieve better security with a stored procedure. Using a stored procedure we can also avoid the SQL Injection attack.

What is an execution plan?

An execution plan is nothing but for the query to retrieve the data what is the best possible way available. This depends on the indexes that available on the SQL Server to help that query. Based on those it generates the execution plan and then it executes the query.

In the next article, I am going to discuss the [SQL Server Stored Procedure Return Value](#) with examples. Here, in this article, I try to explain the **SQL Server Stored Procedure** with Examples. I hope this SQL Server Stored Procedure with Examples article will help you with your need.